

Exercise :

- I. **CheckPassFail (if-else):** Write a program called **CheckPassFail** which prints "PASS" if the int variable "mark" is more than or equal to 50; or prints "FAIL" otherwise. The program shall always print "DONE" before exiting.

Exercise :

- II. **CheckOddEven (if-else):** Write a program called **CheckOddEven** which prints "Odd Number" if the int variable "number" is odd, or "Even Number" otherwise. The program shall always print "BYE!" before exiting.

Hints: n is an even number if $(n \% 2)$ is 0; otherwise, it is an odd number.

Exercise :

- III. **PrintNumberInWord (nested-if, switch-case):** Write a program called **PrintNumberInWord** which prints "ONE", "TWO", ..., "NINE", "OTHER" if the int variable "number" is 1, 2, ..., 9, or other, respectively. Use (a) a "nested-if" statement; (b) a "switch-case" statement.

Exercise :

- IV. **PrintDayInWord (nested-if, switch-case):** Write a program called **PrintDayInWord** which prints "Sunday", "Monday", ... "Saturday" if the int variable "day" is 0, 1, ..., 6, respectively. Otherwise, it shall print "Not a valid day". Use (a) a "nested-if" statement; (b) a "switch-case" statement.
- Modify the program to use a "while-do" loop instead of "for" loop.
 - Modify the program to use a "do-while" loop.
 - What is the difference between "for" and "while-do" loops? What is the difference between "while-do" and "do-while" loops?
 - Modify the program to sum from 111 to 8899, and compute the average. Introduce an int variable called count to count the numbers in the specified range.
 - Modify the program to sum only the *odd* numbers from 1 to 100, and compute the average. (HINTS: n is an odd number if $n \% 2$ is not 0.)
 - Modify the program to sum those numbers from 1 to 100 that is divisible by 7, and compute the average.
 - Modify the program to find the "sum of the squares" of all the numbers from 1 to 100, i.e. $1*1 + 2*2 + 3*3 + \dots + 100*100$.

Exercise :

- V. **Product1ToN (Loop):** Write a program called **Product1ToN** to compute the product of integers 1 to 10 (i.e., $1 \times 2 \times 3 \times \dots \times 10$). Try computing the product from 1 to 11, 1 to 12, 1 to 13 and 1 to 14. Write down the product obtained and explain the results.

- Hints:** Declare an int variable called product (to accumulate the product) and initialize to 1.
- Try:** Compute the product from 1 to 11, 1 to 12, 1 to 13 and 1 to 14. Write down the product obtained and decide if the results are correct.
- Try:** Repeat the above, but use long to store the product. Compare the products obtained.

Hints: Product of 1 to 13 (=6227020800) is outside the range of int [-2147483648, 2147483647], but within the range of long. Take note that computer programs may not produce the correct answer even though everything seems correct!

Exercise :

VI.

Exercise HarmonicSum (Loop): Write a program called **HarmonicSum** to compute the sum of a harmonic series, as shown below, where $n=50000$. The program shall compute the sum from *left-to-right* as well as from the *right-to-left*. Obtain the difference between these two sums and explain the difference. Which sum is more accurate?

$$Harmonic(n) = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$$

Exercise :

VII. **ComputePI (Loop & Condition):** Write a program called **ComputePI** to compute the value of π , using the following series expansion. You have to decide on the termination criterion used in the computation (such as the number of terms used or the magnitude of an additional term). Is this series suitable for computing π ?

$$\pi = 4 \times \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \frac{1}{13} - \frac{1}{15} + \dots \right)$$

JDK maintains the value of π in a double constant called `Math.PI`. Compare the values obtained and the `Math.PI`, in percents of `Math.PI`.

Hints: Add to sum if the denominator modulus 4 is 1, and subtract from sum if it is 3.

Exercise :

VIII. **Fibonacci (Loop):** Write a program called **Fibonacci** to display the first 20 Fibonacci numbers $F(n)$, where $F(n)=F(n-1)+F(n-2)$ and $F(1)=F(2)=1$. Also compute their average. The output shall look like:

The first 20 Fibonacci numbers are:

1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765

The average is 885.5

Ex. IX CheckerBoard (nested-loop): Write a program called **CheckerBoard** that displays the following $n \times n$ ($n=7$) checkerboard pattern using two nested for-loops.

```
# # # # # # #
  # # # # # # #
# # # # # # #
  # # # # # # #
# # # # # # #
  # # # # # # #
# # # # # # #
```

Your program should use only three output statements, one EACH of the followings:

```
System.out.print("# ");    // print # and a space, without newline
System.out.print(" ");     // print a space, without newline
System.out.println();      // print a newline
```

Exercise :

Exercise

X. TimeTable (nested-loop): Write a program called **TimeTable** to produce the multiplication table of 1 to 9 as shown using two nested for-loops:

```
* | 1 2 3 4 5 6 7 8 9
-----
1 | 1 2 3 4 5 6 7 8 9
2 | 2 4 6 8 10 12 14 16 18
3 | 3 6 9 12 15 18 21 24 27
4 | 4 8 12 16 20 24 28 32 36
5 | 5 10 15 20 25 30 35 40 45
6 | 6 12 18 24 30 36 42 48 54
7 | 7 14 21 28 35 42 49 56 63
8 | 8 16 24 32 40 48 56 64 72
9 | 9 18 27 36 45 54 63 72 81
```

Modify the program to print the multiplication table of 1 to 12. (Hints: use `printf()` to format the numbers.)

Exercise Xi KeyboardScanner (Keyboard Input): Write a program called **KeyboardScanner** to prompt user for an `int`, a `double`, and a `String`. The output shall look like (the inputs are shown in bold):

```
Enter an integer: 12
Enter a floating point number: 33.44
Enter your name: Peter
Hi! Peter, the sum of 12 and 33.44 is 45.44
```

Exercise XII

FileScanner (File Input): Write a program called **FileScanner** to read an int, a double, and a String from a text file called "in.txt", and produce the following output:

```
The integer read is 12
The floating point number read is 33.44
The String read is "Peter"
Hi! Peter, the sum of 12 and 33.44 is 45.44
```

You need to create a text file called "in.txt" (in Eclipse, right-click on the "project" ⇒ "New" ⇒ "File") with the following contents:

Exercise Xiii

ReverseString: Write a program called **ReverseString**, which prompts user for a String, and prints the *reverse* of the String. The output shall look like:

```
Enter a String: abcdef
The reverse of the String "abcdef" is "fedcba".
```

Exercise XIV

CheckVowelsDigits: Write a program called **CheckVowelsDigits**, which prompts the user for a String, counts the number of vowels (a, e, i, o, u, A, E, I, O, U) and digits (0-9) contained in the string, and prints the counts and the percentages (with 2 decimal digits). For example,

```
Enter a String: testing12345
Number of vowels: 2 (16.67%)
Number of digits: 5 (41.67%)
```

Hints:

- To check if a char `c` is a digit, you can use boolean expression `(c >= '0' && c <= '9')`; or use built-in boolean function `Character.isDigit(c)`.
- You could use `in.next().toLowerCase()` to convert the input String to lowercase reduce the number of cases.
- To print a % using `printf()`, you need to use `%%`. This is because % has a special meaning in `printf()`, e.g., `%d` and `%f`.

Exercise XV

PhoneKeyPad: On your phone keypad, the alphabets are mapped to digits as follows: ABC(2), DEF(3), GHI(4), JKL(5), MNO(6), PQRS(7), TUV(8), WXYZ(9). Write a program called **PhoneKeyPad**, which prompts user for a String (case insensitive), and converts to a sequence of keypad digits. Use a nested-if (or switch-case) in this exercise.

Hints:

- You can use `in.next().toLowerCase()` to read a `String` and convert it to lowercase to reduce your cases.
- In `switch`, you can handle multiple cases, e.g.,

Exercise 16

GradesAverage (Array): Write a program called **GradesAverage**, which prompts user for the number of students, reads it from the keyboard, and saves it in an `int` variable called `numStudents`. It then prompts user for the grades of each of the students and saves them in an `int` array called `grades`. Your program shall check that the grade is between 0 and 100. A sample session is as follow:

Exercise 17

Write a boolean method called `isOdd()` in a class called `OddTest`, which takes an `int` as input and returns `true` if it is odd. The signature of the method is as follows:

```
public static boolean isOdd(int number)
```

Also write the `main()` method that prompts user for a number, and prints "ODD" or "EVEN". You should test for negative input.

Exercise 18.

Write a boolean method called `hasEight()`, which takes an `int` as input and returns `true` if the number contains the digit 8 (e.g., 18, 808). The signature of the method is as follows:

```
public static boolean hasEight(int number)
```

Exercise 19

Write a program called **MagicSum**, which prompts user for numbers, and produce the sum of numbers containing the digit 8. Your program should use the above methods. A sample output of the program is as follows:

```
Enter a positive integer or -1 to exit: 1
Enter a positive integer or -1 to exit: 2
Enter a positive integer or -1 to exit: 3
Enter a positive integer or -1 to exit: 8
Enter a positive integer or -1 to exit: 88
Enter a positive integer or -1 to exit: -1
The magic sum is: 96
```

Hints: To repeat until input is -1 (called *sentinel* value):

Exercise 20

(Array and Method): Write a method called `printArray()`, which takes an `int` array and print its contents in the form of `{a1, a2, ..., an}`. Take note that there is no comma after the last element. The method's signature is as follows:

```
public static void printArray(int[] array)
```

Also write a test driver to test this method (you should test on empty array, one-element array, and n-element array).

How to handle `double[]` or `float[]`? You need to write another version for `double[]`, e.g.,

```
public static void printArray(double[] array)
public static void printArray(float[] array)
```

The above is known as *method overloading*, where the same method name can have many versions, differentiated by its parameter list.

Exercise 21

(Array and Method): Write a method called `arrayToString()`, which takes an `int` array and return a `String` in the form of `{a1, a2, ..., an}`. Take note that there is no comma after the last element. The method's signature is as follows:

```
public static String arrayToString(int[] array)
```

Also write a test driver to test this method (you should test on empty array, one-element array, and n-element array).

Notes: This is similar to the built-in function `Arrays.toString()`. You could study its source code.

Exercise 22

(Array and Method): Write a boolean method called `contains()`, which takes an array of `int` and an `int`; and returns `true` if the array contains the given `int`. The method's signature is as follows:

```
public static boolean contains(int[] array, int key)
```

Also write a test driver to test this method.

Exercise 23

(Array and Method): Write a method called `search()`, which takes an array of `int` and an `int`; and returns the array *index* if the array contains the given `int`; or `-1` otherwise. The method's signature is as follows:

```
public static int search(int[] array, int key)
```

Also write a test driver to test this method.

Exercise 24

(Array and Method): Write a boolean method called `equals()`, which takes two arrays of `int` and returns `true` if the two arrays are exactly the same (i.e., same length and same contents). The method's signature is as follows:

```
public static boolean equals(int[] array1, int[] array2)
```

Also write a test driver to test this method.

Exercise 25

(Array and Method): Write a boolean method called `copyOf()`, which takes an `int` Array and returns a copy of the given array. The method's signature is as follows:

```
public static int[] copyOf(int[] array)
```

Also write a test driver to test this method.

Write another version for `copyOf()` which takes a second parameter to specify the length of the new array. You should truncate or pad with zero so that the new array has the required length.

```
public static int[] copyOf(int[] array, int newLength)
```

NOTES: This is similar to the built-in function `Arrays.copyOf()`.

Exercise 26

(Array and Method): Write a method called `reverse()`, which takes an array of `int` and reverse its contents. For example, the reverse of `{1,2,3,4}` is `{4,3,2,1}`. The method's signature is as follows:

```
public static void reverse(int[] array)
```

Take note that the array passed into the method can be modified by the method (this is called "*pass by reference*"). On the other hand, primitives passed into a method cannot be modified. This is because a clone is created and passed into the method instead of the original copy (this is called "*pass by value*").

Also write a test driver to test this method.

Hint: You need to use a temp location (an `int`) to swap the first element with the last element, and so on.

Exercise 27

(Array and Method): Write a method called `swap()`, which takes two arrays of `int` and swap their contents if they have the same length. It shall return `true` if the contents are successfully swapped. The method's signature is as follows:

```
public static boolean swap(int[] array1, int[] array2)
```

Also write a test driver to test this method.

Hint: You need to use a temp location (an int) to swap the corresponding elements of the two arrays.

Exercise 28

(Array and Method): Write a program called **PrintChart** that prompts the user to input n non-negative integers and draws the corresponding horizontal bar chart. Your program shall use an int array of length n ; and comprise methods `readInput()` and `printChart()`. A sample session is as follows:

```
Enter number of bars: 4
Enter bar 1 value: 4
Enter bar 2 value: 3
Enter bar 3 value: 5
Enter bar 4 value: 7
```

```
**** (4)
*** (3)
***** (5)
***** (7)
```

Exercise 29

Arithmetic (Command-line arguments): Write a program called **Arithmetic** that takes three command-line arguments: two integers followed by an arithmetic operator (+, -, *, or /). The program shall perform the corresponding operation on the two integers and print the result. For example:

```
java Arithmetic 3 2 +
3+2=5
```

```
java Arithmetic 3 2 -
3-2=1
```

```
java Arithmetic 3 2 /
3/2=1
```

Hints:

The method `main(String[] args)` takes an argument: "an array of String", which is often (but not necessary) named `args`. This parameter captures the command-line arguments supplied by the user when the program is invoked. For example, if a user invokes:

```
java Arithmetic 12345 4567 +
```

The three command-line arguments "12345", "4567" and "+" will be captured in a String array `{"12345", "4567", "+"}` and passed into the `main()` method as the argument `args`. That is,

```
args is: {"12345", "4567", "+"} // args is a String array
args.length is: 3 // length of the array
args[0] is: "12345" // 1st element of the String array
args[1] is: "4567" // 2nd element of the String array
args[2] is: "+" // 3rd element of the String array
```



```
args[0].length() is: 5      // length of 1st String element
args[1].length() is: 4      // length of the 2nd String element
args[2].length() is: 1      // length of the 3rd String element
```

Notes:

- To provide command-line arguments, use the "cmd" or "terminal" to run your program in the form "java *ClassName arg1 arg2*".
- To provide command-line arguments in Eclipse, right click the source code ⇒ "Run As" ⇒ "Run Configurations..." ⇒ Select "Main" and choose the proper main class ⇒ Select "Arguments" ⇒ Enter the command-line arguments, e.g., "3 2 +" in "Program Arguments".
- To provide command-line arguments in NetBeans, right click the "Project" name ⇒ "Set Configuration" ⇒ "Customize..." ⇒ Select categories "Run" ⇒ Enter the command-line arguments, e.g., "3 2 +" in the "Arguments" box (but make sure you select the proper Main class).

Question: Try "java Arithmetic 2 4 *" (in CMD shell and Eclipse/NetBeans) and explain the result obtained. How to resolve this problem?

In Windows' CMD shell, * is known as a wildcard character, that expands to give the list of file in the directory (called Shell Expansion). For example, "dir *.java" lists all the file with extension of ".java". You could double-quote the * to prevent shell expansion. Eclipse has a bug in handling this, even * is double-quoted. NetBeans??

Exercise 30

SumDigits (Command-line arguments): Write a program called **SumDigits** to sum up the individual digits of a positive integer, given in the command line. The output shall look like:

```
java SumDigits 12345
The sum of digits = 1 + 2 + 3 + 4 + 5 = 15
```